

Operating Systems and System Programming

Skill-5

Aim

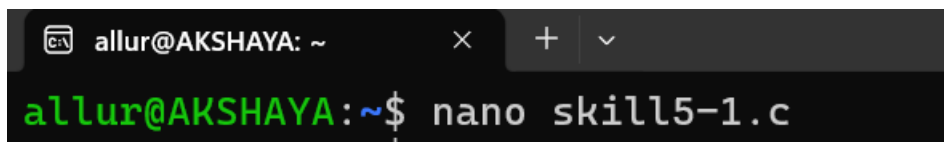
To implement single and double quote handling in a shell, preserve literal content and spaces, support variable expansion, store quoted strings, validate parsing results, and test different inputs.

Algorithm

1. Start the program.
2. Read input from the user.
3. Identify single and double quotes.
4. Preserve the content inside single quotes without variable expansion.
5. Preserve spaces inside double quotes and allow variable expansion.
6. Store the processed strings in a buffer.
7. Display the parsed output.
8. Test different inputs and validate the results.
9. Stop the program.

Task-1:

Step-1:

A screenshot of a terminal window. The title bar shows a window icon, the text 'allur@AKSHAYA: ~', and window control buttons (close, maximize, minimize). The terminal content shows the prompt 'allur@AKSHAYA:~\$' followed by the command 'nano skill5-1.c'.

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill5-1.c *  
  
#include <stdio.h>  
#include <string.h>  
  
int main()  
{  
    char input[100];  
    char output[100];  
    int i, j = 0;  
  
    printf("Enter text: ");  
    fgets(input, sizeof(input), stdin);  
  
    for (i = 0; input[i] != '\0'; i++)  
    {  
        if (input[i] == '\'' )  
            continue;  
  
        output[j++] = input[i];  
    }  
  
    output[j] = '\0';  
    printf("Output: %s", output);  
  
    return 0;  
}  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo
```

Step-3: Compile and Run

```
allur@AKSHAYA:~$ gcc skill5-1.c -o skill5-1  
allur@AKSHAYA:~$ ./skill5-1  
Enter text: 'Hello $USER'  
Output: Hello $USER  
allur@AKSHAYA:~$
```

Explanation: This program removes single quotes and displays their contents literally. It does not expand \$USER.

Task-2:

Step-1:

```
allur@AKSHAYA:~$ nano skill5-2.c
```

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill5-2.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
int main()  
{  
    char input[100];  
    char output[200];  
    int i, j = 0;  
  
    printf("Enter text: ");  
    fgets(input, sizeof(input), stdin);  
  
    for (i = 0; input[i] != '\0'; i++)  
    {  
        if (input[i] == '"')  
            continue;  
  
        if (input[i] == '$')  
        {  
            char name[50];  
            char *value;  
            int k = 0;  
  
            i++;  
            while ((input[i] >= 'A' && input[i] <= 'Z') ||  
                    (input[i] >= 'a' && input[i] <= 'z') ||  
                    (input[i] >= '0' && input[i] <= '9') ||  
                    input[i] == '_')  
            {  
                name[k++] = input[i++];  
            }  
            name[k] = '\0';  
            i--;  
  
            value = getenv(name);  
            if (value != NULL)  
            {  
                strcpy(&output[j], value);  
                j += strlen(value);  
            }  
            else  
            {  
                output[j++] = input[i];  
            }  
        }  
  
        output[j] = '\0';  
        printf("Output: %s", output);  
  
        return 0;  
    }  
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 skill5-2.c *  
    while ((input[i] >= 'A' && input[i] <= 'Z') ||  
            (input[i] >= 'a' && input[i] <= 'z') ||  
            (input[i] >= '0' && input[i] <= '9') ||  
            input[i] == '_')  
    {  
        name[k++] = input[i++];  
    }  
    name[k] = '\0';  
    i--;  
  
    value = getenv(name);  
    if (value != NULL)  
    {  
        strcpy(&output[j], value);  
        j += strlen(value);  
    }  
    else  
    {  
        output[j++] = input[i];  
    }  
}  
  
output[j] = '\0';  
printf("Output: %s", output);  
  
return 0;  
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Un
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line M-E Re

Step-3: Compile and Run

```
allur@AKSHAYA:~$ gcc skill5-2.c -o skill5-2
allur@AKSHAYA:~$ ./skill5-2
Enter text: "Hello $USER"
Output: Hello allur
allur@AKSHAYA:~$
```

Explanation: This program removes double quotes, preserves spaces, and expands environment variables such as \$USER.

Result

Single and double quote handling was successfully implemented. Single quotes preserved literal content without variable expansion, while double quotes preserved spaces and allowed variable expansion. The parsing results were tested and verified successfully.